



Vue.js

프론트엔드 프레임워크 입문 강의

구조적 사고로 UI를 만든다

“ 프레임워크 자체보다 ‘웹 화면이 구조적으로 만들어지는 방식’을 이해합니다 ”



단순히 “프론트엔드 프레임워크를 배운다”



CORE OBJECTIVE

“웹 화면이 구조적으로 어떻게 만들어지는지
이해한다”

프론트엔드 프레임워크가 필요한 이유



PROBLEM



과거 방식의 한계

Vanilla JS + jQuery

HTML / JS / CSS 혼재

코드의 역할이 명확히 분리되지 않아 파일 하나가 비대해지고 관리가 어려움

상태(State) 관리의 복잡성

데이터가 변할 때마다 DOM을 직접 조작해야 해서 버그 발생 확률 폭증

유지보수 비용 증가

화면 규모가 커질수록 어디를 고쳐야 할지 파악하기 힘들어짐

VS

SOLUTION



프레임워크의 역할

Vue.js / React / Angular

컴포넌트 단위 분리

화면을 레고 블록처럼 독립적인 단위로 나누어 재사용성과 조립성을 극대화

데이터 바인딩 자동화

데이터(State)만 바꾸면 화면(UI)이 알아서 변경됨 (Reactive)

확장성 및 협업 효율

표준화된 구조 덕분에 여러 개발자가 동시에 작업하기 용이함

CORE PHILOSOPHY

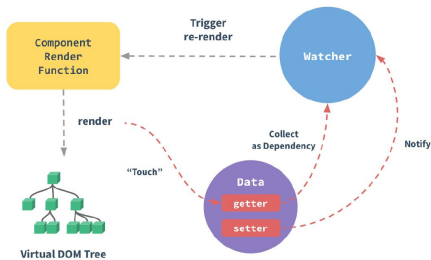
프레임워크는 코드를 줄이기 위한 게 아니라 사고 방식을 정리하기 위한 도구입니다

Vue.js란 무엇인가?

“ 사용자 인터페이스(UI)를 만들기 위한 점진적 프레임워크 ”

— The Progressive JavaScript Framework

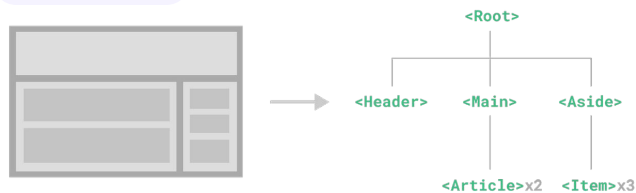
CORE



Reactive

데이터(State)를 변경하면 화면(View)이 자동으로 감지하여 업데이트됩니다. DOM을 직접 조작할 필요가 없습니다.

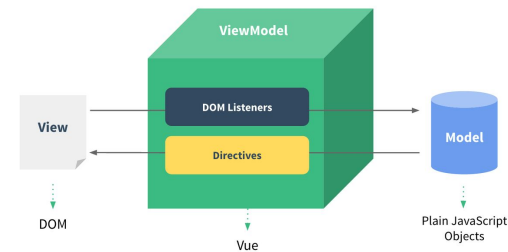
STRUCTURE



Component

화면 전체를 작고 독립적인 블록(Component)으로 분리하여 레고처럼 조립하고 재사용합니다.

PI



Progressive

프로젝트 규모에 따라 필요한 만큼만 도입 가능합니다. CDN 스크립트 하나로도 바로 시작할 수 있습니다.

Vue.js의 핵심 특징

1 직관적인 문법

```
⬢ ⬢ ⬢  
<p>{{ message }} </p>  
<button @click ="count++" >  
+  
</button>
```

HTML 기반 템플릿 기존 웹 개발 지식 (HTML/CSS)을 그대로 활용할 수 있어 익숙합니다.

낮은 진입 장벽 복잡한 설정 없이 스크립트 태그만으로도 바로 시작 가능합니다.

2 Single File Component

```
App.vue  
  
<template> ... </template>  
  
<script> ... </script>  
  
<style> ... </style>
```

관심사의 분리 한 파일 안에 구조 / 로직 / 스타일을 명확히 구분하여 관리합니다.

높은 가독성 "이 파일이 이 화면이다"라는 매칭이 매우 직관적입니다.

3 쉬운 양방향 사고



명확한 연결 고리 data와 template의 연결이 명시적이라 데이터 흐름을 이해하기 쉽습니다.

상태 추적 용이 어디서 데이터가 변해서 화면이 바뀌었는지 추적이 비교적 수월합니다.

Vue.js vs React



구분	 Vue.js	 React
정체성	프레임워크 (Framework)	라이브러리 (Library)
문법 스타일	HTML 기반 템플릿 분리된 구조 (template/script/style)	JSX (JavaScript XML) JS 안에 HTML이 포함된 형태
진입 장벽	☆☆☆ 낮음 (쉬움)	☆☆☆ 상대적으로 높음
사고 방식	선언적 + 직관적	함수형 프로그래밍 (불변성 등)
공식 도구	Router, Pinia(Store) 등 공식 제공	다양한 외부 라이브러리 조합 필요
학습 곡선	 완만함	 초반에 가파름



⚠ “Vue가 쉽고 React가 어렵다”가 아닙니다. “접근하는 출발점(사고방식)이 다르다”는 뜻입니다.

왜 Vue.js를 선택했는가?



EDUCATION

교육 관점의 이유

Learning Curve

기초 지식의 자연스러운 연결

HTML, CSS, JS의 기본 문법을 그대로 사용하므로 기존 웹 지식을 100% 활용 가능

컴포넌트 개념 이해에 최적

직관적인 구조 덕분에 초보자도 '화면을 조립한다'는 개념을 빠르게 체득

빠른 성취감과 낮은 포기율

복잡한 설정 없이도 결과물이 바로 나와 학습 동기 부여가 지속됨

+



INDUSTRY

실무 관점의 이유

Job Market

높은 현업 사용 빈도

중소·중견기업, SI 프로젝트, 사내 어드민 시스템 개발에 유리

비즈니스 로직 중심 개발

데이터 중심의 대시보드, 관리자 페이지 등 실용적인 서비스 구축에 적합

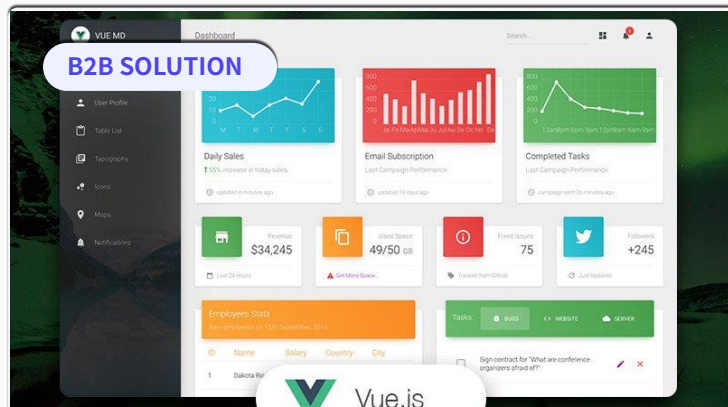
생산성과 유지보수

가독성 높은 코드로 인해 협업이 용이하고 개발 속도가 빠름

Vue로 만들 수 있는 것들

● Real World Applications

단순한 웹페이지를 넘어, 복잡하고 인터랙티브한 애플리케이션을 효율적으로 구축할 수 있습니다.

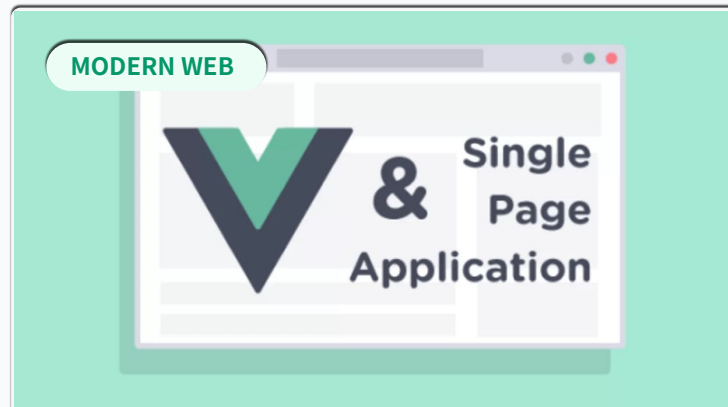


Admin Dashboard

데이터 시각화, 차트, 테이블이 많은 관리자 페이지 구축에 탁월합니다. 컴포넌트 재사용으로 개발 속도가 매우 빠릅니다.

Chart.js

Tables

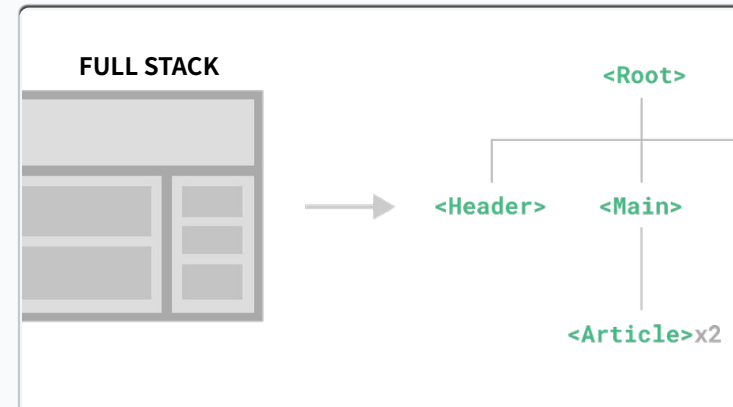


SPA Service

페이지 새로고침 없는 Single Page Application으로 앱(App)처럼 부드럽고 끊김 없는 사용자 경험을 제공합니다.

Vue Router

UX/UI



SaaS & Tools

로그인, 회원가입, 실시간 동기화 등 복잡한 비즈니스 로직이 필요한 SaaS 형태의 웹 서비스를 빠르게 구현합니다.

Firebase

State Mgmt

앞으로의 실습 로드맵 예고

🚩 Final Goal: Portfolio

1. Vue.js 시작

개발환경 설정 및 기본기

2. Vue.js 뼈대

컴포넌트, props, event

3. 상태관리 및 반응형

computed, watch, pinia

4. 중간 예제 실습

TODO 어플리케이션 구현

5. Vue Router

라우팅 및 페이지 관리

6. 실전 활용

API 연동

7. 차트 시각화 및 반응형 UI

데이터 시각화 기법

8. Firebase 연동

auth, firestore

9. 실전 프로젝트

완성된 기술로 실제 애플리케이션 구현